GUI_BUILD_ENV

GUI Build Environment (X11 / GL Dependencies)

Revision: 1 **Last modified:** 2026-07-11T00:00:00Z

Summary

A plain `go build ./...` (or `go test ./...`) from `helix_code/` **fails** on this host (and on any Linux host without X11/OpenGL development headers installed) because several packages transitively depend on Fyne's desktop-GUI stack, which in turn depends on `go-gl/glfw` and `go-gl/gl`, both of which are cgo packages requiring system X11 and OpenGL development headers to compile.

This is not a project bug — it is an environment dependency the project's own `Makefile` already accounts for via a `nogui` build tag (see §3). This document exists so that dependency is no longer undocumented: an agent or operator hitting this failure should reach for `make verify-compile`, not chase a phantom Go-level compile error.

1. What actually fails, and why

Verified on this host (ALT Workstation 11.1) on 2026-07-11:

```
$ cd helix_code && go build ./...
# github.com/go-gl/gl/v2.1/gl
# [pkg-config --cflags  -- gl gl]
Package gl was not found in the pkg-config search path.
Perhaps you should add the directory containing `gl.pc'
to the PKG_CONFIG_PATH environment variable
No package 'gl' found
...
# github.com/go-gl/glfw/v3.3/glfw
In file included from ./glfw/src/internal.h:188,
                 from ./glfw/src/context.c:30,
                 from .../github.com/go-gl/glfw/v3.3/glfw@.../c_glfw.go:4:
./glfw/src/x11_platform.h:33:10: fatal error: X11/Xlib.h: No such file or
   33 | #include <X11/Xlib.h>
```

```
      |           ^~~~~~~~~~~~
compilation terminated.
```

`helix_code/go.mod` pulls in `fyne.io/fyne/v2 v2.7.0` (the Fyne GUI toolkit — see CLAUDE.md §3.1, "UI: Fyne v2.7.0 (desktop GUI)"), which depends transitively on:

- `github.com/go-gl/gl/v2.1/gl` — a cgo binding that resolves its OpenGL headers/link flags via `pkg-config gl` at build time.
- `github.com/go-gl/glfw/v3.3/glfw` — a cgo binding whose C sources (`glfw/src/x11_platform.h` and friends) directly `#include <X11/Xlib.h>` and the sibling Xrandr/Xcursor/Xinerama/Xi/Xxf86vm headers on Linux.

Neither `gl.pc` (the pkg-config metadata for OpenGL) nor `X11/Xlib.h` are present by default on a minimal ALT Linux install — confirmed on this host: `pkg-config --exists gl` fails, and `/usr/include/X11/Xlib.h` does not exist.

Packages affected in this tree (i.e. packages that import Fyne or go-gl, directly or transitively, and therefore require these headers to build with the default build tags):

- `applications/desktop/` — the Fyne desktop GUI application. Its own `main.go` / `main_nogui.go` split (see §3) is exactly the accommodation for this dependency.
- `applications/aurora_os/` — imports the same Fyne stack (`theme.go`, `main_nogui.go` present here too).
- `applications/harmony_os/` — same dependency chain (`distributed.go`, i18n bundle files).
- `tests/ui/` — `desktop_widget_test.go` and `ui_harness.go` reference `fyne.io` directly, so `go test ./tests/ui/...` without the `nogui` tag hits the same wall.

2. Installing the headers (operator action — needs root/pkg-manager)

This is an **operator action**, not something an AI agent should do autonomously in a shared/production-adjacent environment — installing system packages requires root and mutates host state outside this repository's control (per §11.4.133's spirit: changes outside the target project's own tree, made with elevated privilege, are an operator decision). The commands below are documented for the operator's convenience, not executed by any automation in this repo.

ALT Linux (this host's distro — ALT Workstation 11.1, apt-rpm)

Package names verified present in the `apt-cache search` index on this host on 2026-07-11 (verification means the package NAMES exist in the configured repos — it does NOT mean they were installed or that installing them was tested end-to-end; that step still requires root and was not performed by this task):

```
sudo apt-get update
sudo apt-get install libGL-devel libX11-devel \
    libXrandr-devel libXcursor-devel libXinerama-devel \
    libXi-devel libXxf86vm-devel pkg-config
```

- `libGL-devel` — "Development files for Mesa Library" — provides `gl.pc` for `pkg-config gl`.
- `libX11-devel` — "X11 Libraries and Header Files" — provides `X11/Xlib.h`.
- `libXrandr-devel` / `libXcursor-devel` / `libXinerama-devel` / `libXi-devel` / `libXxf86vm-devel` — the standard sibling X11 extension headers GLFW's Linux backend also needs (RandR, Cursor, Xinerama, XInput2, XF86VidMode). **UNCONFIRMED:** this task verified only that `libGL-devel` and `libX11-devel` are the two packages actually missing on this host and blocking the build (per the exact compiler error above); the five X11-extension `-devel` packages are included because they are GLFW's documented standard Linux dependency set, but their necessity was not individually re-proven by triggering their specific missing-header errors on this host.
- `pkg-config` — already installed on this host (`pkg-config-0.29.2-alt3.x86_64` via `rpm -q pkg-config`); listed here only for hosts where it is not yet present.

Debian/Ubuntu (apt) — for reference, other supported dev hosts

Not this host's distro; package names below follow the standard Debian/Ubuntu GLFW build-dependency set and were **not** individually verified against a live apt index in this task:

```
sudo apt-get install libgl1-mesa-dev libx11-dev \
    libxrandr-dev libxcursor-dev libxinerama-dev \
    libxi-dev libxxf86vm-dev pkg-config
```

Fedora/RHEL (dnf) — for reference

```
sudo dnf install mesa-libGL-devel libX11-devel \
    libXrandr-devel libXcursor-devel libXinerama-devel \
    libXi-devel libXxf86vm-devel pkgconfig
```

macOS / Windows

Fyne's desktop backend uses the platform's native OpenGL framework (Cocoa/OpenGL.framework on macOS, WGL on Windows) via cgo and the platform C toolchain (Xcode Command Line Tools / MSVC or MinGW), not X11 — the X11-specific failure documented in §1 is Linux-only. See the upstream Fyne "Getting Started" docs for the platform prerequisites if building the GUI target on those platforms.

3. The sanctioned no-X11 path: `make verify-compile`

The project's own `helix_code/Makefile` already has the accommodation for hosts without these headers — a `nogui` build tag:

```
verify-compile:
    @echo "   Verifying code compilation (nogui — no X11 system
        libs required)..."
    @$(GO) build -tags=nogui ./... && echo "   All packages
        compile successfully"
```

i.e. `make verify-compile` is exactly `go build -tags=nogui ./...`. Verified GREEN on this host on 2026-07-11 (no X11/GL headers installed):

```
$ make verify-compile
   Verifying code compilation (nogui — no X11 system libs required)...
   All packages compile successfully
```

The `nogui` tag works by build-tag-gating an alternate, headless entry point in each GUI-touching package — e.g. `applications/desktop/main_nogui.go`, `applications/desktop/specify_nogui.go`, `applications/desktop/generate_nogui.go`, `applications/aurora_os/main_nogui.go` — so the package compiles without ever importing the Fyne/go-gl chain. The Makefile also exposes this explicitly as its own target:

```
make desktop-nogui   # builds bin/helix-desktop-cli via -tags
        nogui
```

Use `make verify-compile` (or `go build -tags=nogui ./...` / `go test -tags=nogui ./...`) as the default dev/CI-equivalent compile check on any host that does not have the X11/GL headers installed. Only reach for a bare `go build ./...` / `go test ./...` (no tags), or `make desktop` / `make desktop-linux` (which build the real GUI binary), on a host where §2's headers are confirmed installed.

4. This is the §11.4.77 regeneration mechanism for this dependency

Per Constitution §11.4.77 (regeneration-mechanism-required mandate): any artefact or environment dependency the project relies on but does not version-control must have a documented, non-interactive mechanism to re-obtain or re-generate it. The X11/OpenGL development headers are exactly such a dependency:

- **Not versioned, and correctly so** — these are host operating-system packages (shared libraries + C headers), not project source; they are never meant to live in this git tree, and no `.gitignore` entry is needed for them since nothing here ever writes them into the tree.
- **Re-obtained via the OS package manager** — §2 above IS the regeneration mechanism: `apt-get install libGL-devel libX11-devel ...` (or the distro-appropriate equivalent) deterministically restores the missing dependency on any fresh host, from the distro's own package index — nothing project-specific needs to be rebuilt or regenerated.
- **The honest fallback is `make verify-compile`** — a host where the operator has not yet run §2 (or cannot, e.g. a locked-down CI runner) is not blocked from verifying the codebase compiles: `-tags nogui` is the documented, always-available fallback that requires zero system packages, exercising every package's headless code path instead of skipping the check entirely.
- **Honest boundary:** `make verify-compile` (nogui) proves the headless/CLI code paths compile. It does **not** prove the actual GUI binary (`make desktop` / `make desktop-linux`, which link against real Fyne/go-gl) builds or runs — that still requires the operator to install the §2 headers on a host that needs to produce or test the GUI artifact. This document does not claim a full GUI build was executed or passed on this host; only `make verify-compile` was run and confirmed green here (§3).

Sources verified 2026-07-11: helix_code/go.mod, helix_code/Makefile (verify-compile / desktop-nogui / desktop targets), live `go build ./...` output on this host (ALT Workstation 11.1), live `apt-cache search` package-name lookups on this host, `pkg-config --exists gl` + `/usr/include/X11/Xlib.h` presence checks on this host, `rpm -q pkg-config` on this host.